BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Midterm
Duration: 80 minutes

Set - A

Semester: Summer 2024
Full Marks: 40

CSE 420: Compiler Design

Figures in the right margin indicate marks.

Answer all the questions

COs	Questions	Marks
CO3	1. Consider the following grammar and look at the SLR(1) parse table below: 1. $E \rightarrow E + T$ 2. $E \rightarrow T$ 3. $T \rightarrow T * F$ 4. $T \rightarrow F$ 5. $F \rightarrow (E)$ 6. $F \rightarrow id$	10

STATE	ACTION						GOTO		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

	Show the parsing simulation using **stack** for the input string, (***id*id)+id***	
CO3	2. Consider the following *SLR Grammar*. Draw **LR(0) automaton** and construct **SLR parse table** 1. $T \rightarrow B\ C$ 2. $B \rightarrow$ ***int*** 3. $B \rightarrow$ ***float*** 4. $C \rightarrow \varepsilon$ 5. $C \rightarrow$ [**num]** C	10
CO2	3. Construct the DFA from the following RE using **Direct Method** (a | ε) (a | b | c) (b | c)*	15
CO1	4. What are different types of errors that can be detected during the compilation process? Which states of the compiler detect what errors?	5

BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Midterm | Semester: Summer 2024
Duration: 80 minutes | Full Marks: 40

Set - B

CSE 420: Compiler Design

Figures in the right margin indicate marks.

Answer all the questions

COs	Questions	Marks
CO3	1. Consider the following grammar and look at the SLR(1) parse table below:	10

7. $A \rightarrow A + B$
8. $A \rightarrow B$
9. $B \rightarrow B * C$
10. $B \rightarrow C$
11. $C \rightarrow (A)$
12. $C \rightarrow id$

STATE	ACTION						GOTO		
	id	+	*	(	)	$	A	B	C
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

	Show the parsing simulation using **stack** for the input string, ***id+(id*id)***	
CO3	2. Consider the following *SLR Grammar*. Draw **LR(0) automaton** and construct **SLR parse table** 1. $V \rightarrow P\ Q$ 2. $P \rightarrow$ ***int*** 3. $P \rightarrow$ ***float*** 4. $Q \rightarrow \varepsilon$ 5. $Q \rightarrow$ [**size]** Q	10
CO2	3. Construct the DFA from the following RE using **Direct Method** (x | ε) (x | y | z) (y | z)*	15
CO1	4. Explain how lexical analyzer and the syntax analyzer interact during the compilation process	5

Set A

Ans to question no 01:

Stack	Symbol	input buffer	Action
0	$	(id * id) + id $	s4
04	$(	id * id) +id $	s5
045	$(id	* id) +id $	Reduce by F → id
043	$(F	* id) +id $	Reduce by T → F
042	$(T	* id) +id $	s7
0427	$(T*	id) + id $	s5
04275	$(T * id	) +id $	Reduce by F → id
0427 "10"	$(T * F	) +id $	Reduce by T → T*F
042	$(T	) +id $	Reduce by E → T
048	$(E	) +id $	s11
048 "11"	$(E)	+ id $	Reduce by F → (E)
03	$F	+id $	Reduce by T → F
02	$T	+id $	Reduce by E → T
01	$E	+id $	s6
016	$E+	id $	s5
0165	$E + id	$	Reduce by F → id
0163	$E+F	$	Reduce by T → F
0169	$E +T	$	Reduce by E → E +T
01	$ E	$	Accept

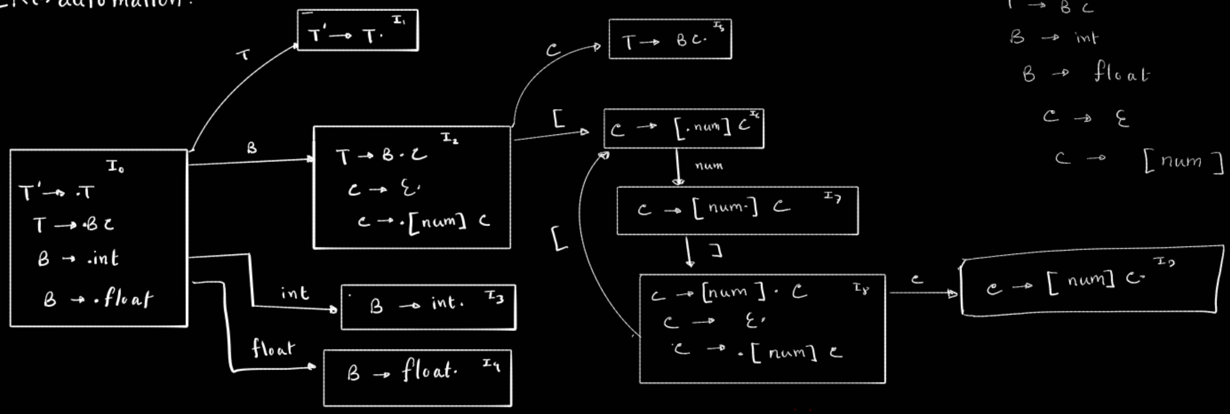
T' → T
T → B C
B → int
B → float
C → ε
C → [num] C

Answer to the question no 2:

Augmentation:

T' → T
T → B C
B → int
B → float
C → ε
C → [num] C.

LR(0) automation:



First (T) = { int, float }
First (B) = { int, float }
First (C) = { [, ε }
Follow (T) = { $ }
Follow (B) = { [, $ }
Follow (C) = { $ }

T → B C (1)
B → int (2)
B → float (3)
C → ε (4)
C → [num] C (5)

state	Action						Go - To		
	[	num	]	int	float	$	T	B	C
0				S3	S4		1	2	
1						Accept			
2	S6					R4			5
3	R2					R2			
4	R3					R3			
5						R1			
6		S7							
7			S8						
8	S6					R4			9
9						R5			

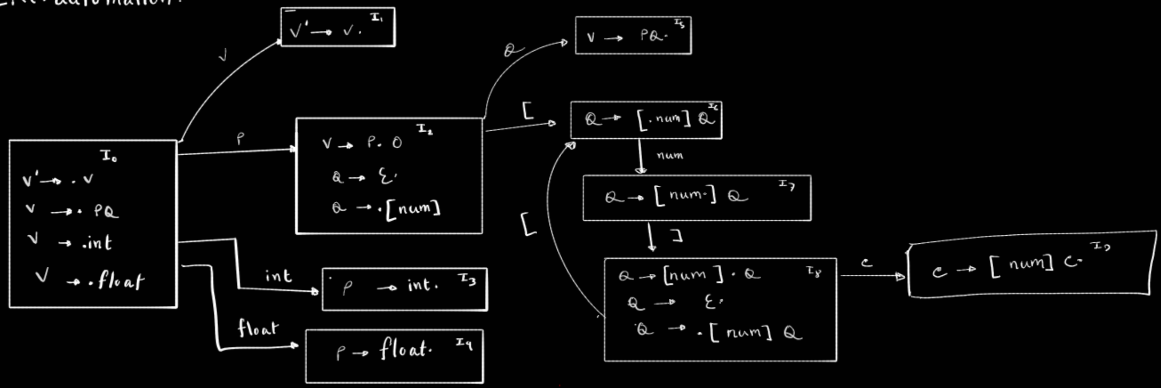
Set B

Answer to the question no 1 :

Stack	Symbol	input buffer	Action
0	$	id + (id * id) $	s5
05	$ id	+ (id * id) $	Reduce by c → id
03	$ C	+ (id * id) $	Reduce by B → c
02	$ B	+ (id * id) $	Reduce by A → B
01	$ A	+ (id * id) $	s6
016	$ A +	(id * id) $	s4
0164	$ A + (	id * id) $	s5
01645	$ A + (id	* id) $	Reduce by c → id
01643	$ A + (c	* id) $	Reduce by B → c
01642	$ A + (B	* id) $	s7
016427	$ A + (B *	id) $	s5
0164275	$ A + (B * id	) $	Reduce by c → id
016427 "10"	$ A + (B * c	) $	Reduce by B → B*c
01642	$ A + (B	) $	Reduce by A → B
01648	$ A + (A	) $	s11
01648 "11"	$ A + (A)	$	Reduce by c → (A)
0163	$ A + c	$	Reduce by B → c
0169	$ A + B	$	Reduce by A → A+B
01	$ A	$	Accept

Answer to the question no 02 :

LR(0) automation:



Augmentation

V' → V

V → PQ

P → int

P → float

Q → ε

Q → [num] Q

First (V) = { int, float }

First (P) = { int, float }

First (Q) = { [, ε }

Follow (V) = { $ }

Follow (P) = { [, $ }

Follow (Q) = { $ }

V → PQ (1)

P → int (2)

P → float (3)

Q → ε (4)

Q → [num] Q (5)

state	Action						Go - To		
	[	num	]	int	float	$	V	P	Q
0				S3	S4		1	2	
1						Accept			
2	S6					R4			5
3	R2					R2			
4	R3					R3			
5						R1			
6		S7							
7			S8						
8	S6					R4			9
9						R5			